

Build: Scenario Engine / Adaptive Scenario Intelligence (ASI)

Objective

Create a deterministic **Scenario Engine** with **Adaptive Scenario Intelligence (ASI)** that answers:

“What happens if the business changes X, Y, Z? Which actions are safest, highest impact, and most feasible?”

It reuses outputs from:

- Viability Engine
- Survival Engine
- Stability Engine
- Growth Engine
- Fragility Index
- Master Aggregator

It takes proposed scenario changes as input and produces:

- Simulated engine outputs
- Updated fragility
- Risk delta
- Recommendations ranking
- Adaptive triggers
- Interpretation for narratives and PDF reports

Recommended Folder

`/src/lib/engines/scenario`

Suggested structure:

```
/src/lib/engines/scenario
├─ types.ts
├─ rules.ts
├─ scenario-engine.ts
├─ scenario-engine.test.ts
└─ index.ts
```

1. Input Contract

```
export interface ScenarioEngineInput {
  businessId: string;

  baseState: {
    viability: ViabilityEngineOutput;
    survival: SurvivalEngineOutput;
    stability: StabilityEngineOutput;
    growth: GrowthEngineOutput;
    fragility: FragilityEngineOutput;
    master: MasterAggregatorOutput;
  };

  scenarioChanges: Array<{
    entityType: string; // offering, revenue, expense, capacity,
customer, etc.
    entityId?: string;
    fieldName: string;
    oldValue?: number | string;
    newValue: number | string;
    changeUnit?: string;
    changeDirection?: "increase" | "decrease" | "replace";
    changePct?: number;
  }>;

  metadata?: {
    source?: "user" | "asi" | "system";
    rulesetVersion?: string;
    confidenceLevel?: "LOW" | "MEDIUM" | "HIGH";
```

```
};  
}
```

2. Output Contract

```
export interface ScenarioEngineOutput {  
  engine: "scenario";  
  version: string;  
  businessId: string;  
  
  baseStateSnapshot: any;  
  simulatedStateSnapshot: any;  
  
  simulatedEngines: {  
    viability: ViabilityEngineOutput;  
    survival: SurvivalEngineOutput;  
    stability: StabilityEngineOutput;  
    growth: GrowthEngineOutput;  
    fragility: FragilityEngineOutput;  
    master: MasterAggregatorOutput;  
  };  
  
  deltas: {  
    scoreDelta: {  
      viability?: number;  
      survival?: number;  
      stability?: number;  
      growth?: number;  
      businessIntelligence?: number;  
      fragilityIndex?: number;  
    };  
    riskDelta: Array<{  
      code: string;  
      change: "added" | "removed" | "updated";  
      previousSeverity?: string;  
      newSeverity?: string;  
    }>;  
  };  
}
```

```

    recommendationDelta: Array<{
      type: string;
      change: "added" | "removed" | "updated";
      previousPriority?: string;
      newPriority?: string;
    }>;
  };

  recommendedDecisionPath: {
    scenarioId?: string;
    rankingScore: number;
    reason: string;
    recommendedActions: string[];
  };

  asiTriggers: Array<{
    triggerCode: string;
    triggerType: "risk" | "opportunity" | "trend";
    severity: "LOW" | "MEDIUM" | "HIGH" | "CRITICAL";
    reason: string;
  }>;

  interpretation: {
    headline: string;
    scenarioSummary: string;
    topRisks: string[];
    expectedImpactSummary: string;
    strategicPriority: string;
    nextBestActions: string[];
  };

  confidence: {
    level: "LOW" | "MEDIUM" | "HIGH";
    note: string;
  };

  trace: Record<string, unknown>;
}

```

3. Scenario Application Logic

1. Clone **base state** of all engines (do not mutate live ontology)
2. Apply **scenarioChanges** to cloned state
 - a. For example:
 - i. Increase price by 10%
 - ii. Reduce fixed cost by 15%
 - iii. Delay hiring 2 months
 - iv. Increase delivery capacity by 20%
3. Recompute all engines deterministically using shared **Computation Library**
 - a. Viability
 - b. Survival
 - c. Stability
 - d. Growth
4. Recompute **Fragility Index**
5. Recompute **Master Aggregator**
6. Calculate **deltas** vs base state
7. Generate **recommendation ranking** considering:
 - a. Survival improvement
 - b. Fragility reduction
 - c. Growth opportunity
 - d. Feasibility
 - e. Confidence
8. Generate **ASI triggers** if any dimension exceeds thresholds
9. Generate **interpretation** for narrative and report consumption

4. Scoring / Ranking Logic

Step 1 — Score Impact

Compute weighted delta:

```
deltaScore =  
(0.30 * viabilityDelta) +  
(0.25 * survivalDelta) +
```

```
(0.20 * stabilityDelta) +  
(0.25 * growthDelta)
```

Step 2 — Fragility Reduction

Compute reduction of fragility index from base to simulated state

```
fragilityDelta = baseFragilityIndex - simulatedFragilityIndex
```

Step 3 — Feasibility

Check if scenario violates:

- capacity
- cash runway
- operational limits

If any violation, reduce rankingScore

```
if capacityOverload || runway < threshold ||  
operationalCriticalDependency:  
    rankingScore -= 20
```

Step 4 — Confidence Weight

Multiply rankingScore by average confidence of affected engines

```
rankingScore *= averageConfidence
```

5. Interpretation Logic

Example:

“Increasing price by 10% improves contribution margin and viability by 5 points but reduces demand slightly, causing a minor increase in revenue volatility. Overall, the best path is to reduce fixed cost by 15% before applying price change.”

6. Recommendations

- Automatically prioritise top 3 scenarios by **rankingScore**
- Include ASI triggers for high fragility or survival risks
- Return recommended actionable steps

7. Unit Tests

- Scenario: Price Increase
- Scenario: Reduce Fixed Cost
- Scenario: Increase Capacity
- Scenario: Delay Hiring
- Scenario: Combination (price + cost + capacity)
- Scenario: Violates constraints (e.g., negative runway)
- Scenario: Fragility reduction validation

Expected:

- Recomputed engine outputs
- Deltas calculated correctly
- Recommended decision path selected properly
- ASI triggers generated for risks
- Interpretation string includes meaningful summary

8. Acceptance Criteria

- Scenario Engine accepts input contract
- Simulates all engines deterministically
- Returns structured ScenarioEngineOutput
- Computes scoreDelta, riskDelta, recommendationDelta correctly
- Identifies recommendedDecisionPath

- Generates ASI triggers
- Generates interpretation and confidence
- No mutation of live ontology
- Unit tests pass

9. Codex Prompt

Build the EnterprateAI Scenario Engine with Adaptive Scenario Intelligence in /src/lib/engines/scenario.

The engine must be pure deterministic TypeScript module. No database, API, or LLM calls.

It should accept ScenarioEngineInput containing:

- baseState (outputs from viability, survival, stability, growth, fragility, master aggregator)
- scenarioChanges (price, cost, capacity, hiring, demand, etc.)

It should return ScenarioEngineOutput.

Steps:

1. Clone base state
2. Apply scenario changes
3. Recompute all engines using Computation Library
4. Recompute Fragility Index
5. Recompute Master Aggregator
6. Calculate deltas vs base state
7. Rank scenarios considering survival, fragility, growth, feasibility, confidence
8. Generate ASI triggers
9. Generate interpretation for narratives
10. Return top-ranked decision path with actionable recommendations
11. Include trace object

Create:

- types.ts
- rules.ts

- scenario-engine.ts
- scenario-engine.test.ts
- index.ts

Add unit tests for:

- single variable changes
- multiple variable combination
- constraint violations
- fragility and survival edge cases

Export the engine from index.ts.